

Terminology

In order to better describe the underlying concepts and functionality of UEC, it would be best to define a set of common terminologies. Given below are some of the terms used in this article, along with their definitions:

- **Node:** A node refers to a single server or physical machine.
- **Virtual resource:** This is a proxy or abstraction for the real resource. The virtual resource has the same interface and functions as the real resource, but with different attributes. Virtual resources include virtual CPUs, memory, disks and the network.
- **Server virtualisation:** Server virtualisation turns a node into a set of virtual machines (VM), with each VM appearing to have its own set of virtual resources, such as a virtual CPU, memory, disk and network.
- **Hypervisor or a Virtual Machine Monitor (VMM):** A hypervisor or virtual machine monitor is a layer of software that virtualises a node into a set of virtual machines. The hypervisor is responsible for the management of the virtual machines.
- **Virtual Machine (VM):** This is an isolated working environment with its own set of virtual resources, including the CPU, memory, disk and the network. A VM can run an OS inside it. On a given node, it is possible to run one or more VMs.
- **Guest OS:** The guest OS is the operating system that runs within the virtual machine.
- **Domain:** A domain is an instance of an operating system running on a virtual machine. The terms 'domain' and 'virtual machines' are often used interchangeably.
- **Virtual Machine Image:** A virtual machine image, or simply the image, is a file on the hard disk that can be used to create an instance of the virtual machine.
- **Volume:** A volume is either a block device, a raw file or a special format file.
- **Pool:** This provides a means of taking a chunk of storage and dividing it into volumes. A pool can be used to manage a physical disk, NFS server, iSCSI target, host adapter and LVM group.
- **Full virtualisation:** In a full virtualisation solution, the hypervisor must intercept privileged instructions from the guest OS, and these are then simulated by the hypervisor to fulfil the request on the CPU hardware. Simulating instructions inside the hypervisor causes performance degradation. In a full virtualisation solution, no modifications are needed to the guest OS.
- **Para virtualisation:** In a para virtualisation solution, privileged instructions can be run directly against the CPU hardware. The guest OS must be modified in order to cooperate with the underlying hypervisor. A para virtualisation solution is faster than full virtualisation, since no simulation of instructions by the hypervisor is required.

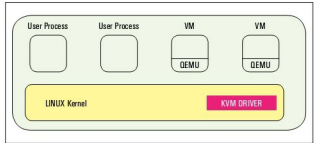


Figure 1: The KVM architecture

- **Hardware-assisted virtualisation:** A hardware-assisted virtualisation solution requires that the underlying CPU hardware provide support for virtualisation. This approach combines the benefits of full virtualisation and para virtualisation. No modification is needed to the guest OS, and privileged instructions are executed by the CPU hardware.

KVM

The two dominant virtualisation technologies in the Linux world are Xen and KVM (Kernel-based Virtual Machine)—both being open source. Xen is a para virtualisation-based hypervisor, while KVM is a hardware-assisted virtualisation-based hypervisor. UEC is based on KVM.

KVM is a hardware-assisted virtualisation solution for Linux on x86 hardware containing virtualisation extensions (Intel VT or AMD-V). KVM adds virtualisation support directly to the Linux kernel. It was introduced to the Linux kernel version 2.6.20—the current kernel version is 2.6.31.

KVM is essentially only a loadable kernel module (*kvm.ko*) that provides the core virtualisation infrastructure. This is besides the processor-specific module—*kvm-intel.ko* or *kvm-amd.ko*, depending on whether it's an Intel or an AMD processor, respectively. KVM also requires a modified QEMU that runs in user space (we'll discuss this later). KVM itself is responsible for virtualising the memory, while QEMU is responsible for virtualising the I/O.



Note: QEMU is an open source processor emulator. All I/O requests (disk and network I/O) made by the guest OS are intercepted and routed to be emulated by QEMU.

KVM requires virtualisation support from the underlying processor hardware. In the case of Intel processors, Intel-VT capable processors provide virtualisation support (shows in */proc/cpuinfo* as the *vmx* flag). In the case of AMD processors, AMD-V capable processors provide virtualisation support (shows in */proc/cpuinfo* as the *svm* flag). See the *Reference* section at the end of this article for a